



Implementação de Algoritmos de Busca em Sistemas P2P

Computação Distribuída
Prof. Nabor C. Mendonça



Introdução (I)

Os algoritmos de **busca por inundação** e **passeio aleatório** são técnicas utilizadas para localizar recursos em sistemas peer-to-peer (P2P) não estruturados. A motivação por trás desses algoritmos é a necessidade de localizar recursos de maneira eficiente em uma rede descentralizada, onde a informação não é organizada de maneira hierárquica ou estruturada. Essa necessidade surge das características intrínsecas dos sistemas P2P, que se caracterizam pela ausência de um servidor central, distribuição geográfica ampla e alta volatilidade dos nós.

O algoritmo de busca por inundação, como o próprio nome sugere, envolve o envio de uma requisição de pesquisa a todos os nós vizinhos na rede. Esses nós, por sua vez, retransmitem a solicitação a seus vizinhos e assim por diante, até que a informação seja encontrada ou um limite de hop seja alcançado. Embora esse método seja bastante simples e possa alcançar todos os nós da rede, ele pode ser ineficiente e gerar uma quantidade significativa de tráfego de rede, principalmente em redes P2P de grande escala.



Introdução (II)

Por outro lado, o algoritmo de busca por passeio aleatório envolve escolher um nó vizinho aleatoriamente e enviar a requisição de pesquisa apenas para esse nó, que então repete o processo. Isso continua até que o recurso seja encontrado ou um limite da rede seja alcançado. Este método é mais eficiente em termos de tráfego de rede em comparação com a busca por inundação, mas pode levar mais tempo para encontrar o recurso se ele estiver localizado longe do nó inicial na rede.

Ambos os algoritmos podem ser estendidos para limitar o alcance da busca com um parâmetro **Time to Live (TTL)**, que limita o número de "saltos" que a requisição pode fazer na rede. O TTL é decrementado a cada salto que a requisição faz. Quando o TTL chega a zero, a requisição é descartada e não é mais retransmitida. Isso permite limitar a quantidade de tráfego gerado pela busca por inundação, mas também pode reduzir a probabilidade de encontrar o recurso se ele estiver localizado longe do nó que iniciou a busca.



Introdução (II)

Outra variação que pode ser feita em ambos os algoritmos, também conhecida como **busca informada**, é cada nó manter um **cache local** contendo informações sobre a localização do recursos na rede. O cache de cada nó é atualizado toda vez que a informação sobre a localização de um recurso chegar ao conhecimento do nó. Dessa forma, buscas futuras por recursos cuja localização já é conhecida de outros nós na rede poderão ser aceleradas, sem a necessidade de esperar as mensagens de busca chegarem até os nós que contêm os recursos sendo buscados.



Instruções

Organizados em grupos de 3-4 alunos, vocês deverão:

1. Implementar um programa que, a partir de um arquivo de configuração dado como entrada, cria uma estrutura de dados representando uma rede P2P não estruturada e permite realizar buscas por recursos mantidos pelos nós da rede utilizando diferentes algoritmos de busca (por exemplo, busca por inundação, busca por passeio aleatório, e busca informada).
2. Realizar testes comparando os diferentes algoritmos de busca em diferentes topologias e configurações de redes P2P. As comparações devem ser feitas em termos do número total de mensagens trocadas na rede até que a localização do recurso buscado seja informada ao nó que iniciou a busca ou a busca termine sem encontrar o recurso.



Entregável

Apresentação de slides, compartilhada no Google Drive, contendo:

1. Identificação dos membros de cada equipe.
2. Descrição das principais funcionalidades do programa implementado.
3. Tabelas/gráficos com os resultados dos testes comparativos.

Além dos slides, a equipe deverá demonstrar as principais funcionalidades do programa "ao vivo", executando-o para diferentes topologias e configurações da rede p2p



Requisitos (I)

O programa deverá receber como entrada um arquivo no seguinte formato (ou em um formato equivalente em JSON ou YAML):

```
num_nodes: 12
min_neighbors: 2
max_neighbors: 4
resources:
    n1: r1, r2, r3
    n2: r4, r5
    ...
edges:
    n1, n2
    n1, n3
    n2, n4
    ...
```



Requisitos (II)

Após ler o arquivo de entrada, o programa deverá fazer as seguintes verificações:

1. A rede não pode estar particionada (ou seja, deve existir pelo menos um caminho conectando qualquer nó a qualquer outro nó).
2. Os números mínimo e máximo de vizinhos de cada nó devem obedecer os limites estabelecidos nos parâmetros `min_neighbors` e `max_neighbors`.
3. Não pode haver nós sem recursos.
4. Não pode haver arestas de um nó para ele mesmo.



Requisitos (III)

Uma vez validado o arquivo de configuração, o programa deverá permitir realizar operações de busca pelos recursos mantidos na rede. Cada operação de busca deverá receber os seguintes parâmetros de entrada:

`node_id` – identificador do nó que inicia a busca

`resource_id` – identificador do recurso a ser buscado

`ttl` – valor do TTL da busca

`algo` – algoritmo de busca a ser utilizado (os valores possíveis para este parâmetro são `flooding`, `informed_flooding`, `random_walk`, `informed_random_walk`)

Ao final de cada execução de uma operação de busca, o programa deverá informar o *número total de mensagens* trocadas entre os nós e o *número total de nós* envolvidos na busca



Requisitos (IV)

Opcionalmente, o programa deverá:

- Exibir uma *representação gráfica da rede p2p* criada a partir do arquivo de configuração lido e validado
- Criar uma *animação da rede p2p* ilustrando, em tempo real, os nós envolvidos na busca e as mensagens trocadas entre eles